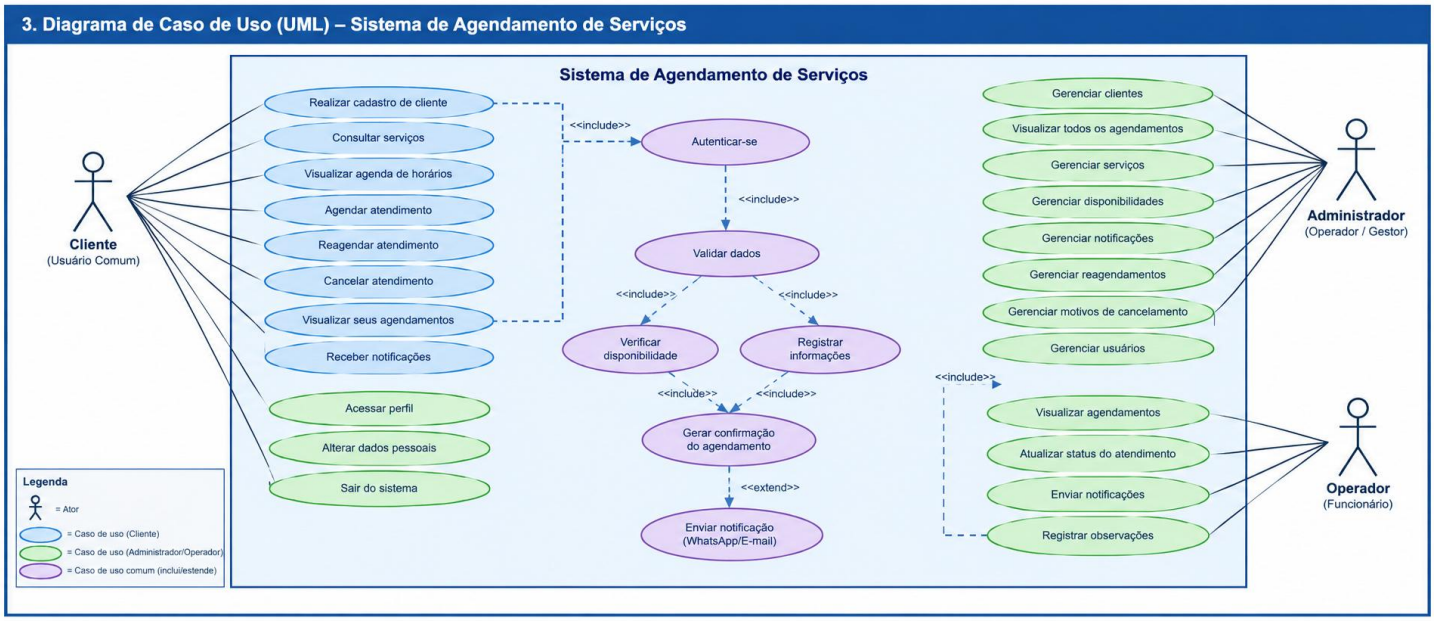
Diagrama de Caso de Uso (UML)



Descrição do diagrama

O Diagrama de Caso de Uso representa as principais interações dos usuários com o Sistema de Agendamento de Serviços.

1. Cliente — Usuário Comum

O **Cliente** é o usuário que utiliza o sistema para realizar e acompanhar seus próprios agendamentos.

Ele pode:

- **Realizar cadastro de cliente** — preencher as informações solicitadas pelo sistema.
- **Consultar serviços** — visualizar os serviços disponíveis.
- **Visualizar agenda de horários** — verificar os horários disponíveis.
- **Agendar atendimento** — escolher um serviço, data e horário.
- **Reagendar atendimento** — alterar um agendamento já realizado.
- **Cancelar atendimento** — cancelar um agendamento.
- **Visualizar seus agendamentos** — consultar seus próprios horários marcados.
- **Receber notificações** — receber informações relacionadas ao atendimento.
- **Acessar perfil** — visualizar seus dados.
- **Alterar dados pessoais** — atualizar informações do perfil.
- **Sair do sistema** — encerrar a sessão.

2. Administrador — Operador/Gestor

O **Administrador** possui um nível maior de acesso. Ele é responsável pela configuração e gerenciamento do sistema.

Pode:

- **Gerenciar clientes** — consultar e administrar os clientes.
- **Visualizar todos os agendamentos** — acompanhar os agendamentos de todos os clientes.
- **Gerenciar serviços** — cadastrar, alterar ou remover serviços.
- **Gerenciar disponibilidades** — definir dias e horários disponíveis.
- **Gerenciar notificações** — controlar as notificações enviadas.
- **Gerenciar reagendamentos** — acompanhar e administrar solicitações de reagendamento.
- **Gerenciar motivos de cancelamento** — cadastrar os motivos que poderão ser utilizados nos cancelamentos.
- **Gerenciar usuários** — criar e administrar os usuários que terão acesso administrativo.

A ideia de **"Administrador (Operador/Gestor)"** é importante porque permite que, no futuro, existem vários funcionários utilizando o sistema, sem precisar criar um sistema diferente para cada pessoa.

3. Operador — Funcionário

O **Operador** é um usuário com acesso às funções operacionais do dia a dia, mas sem necessariamente possuir todas as permissões do administrador.

Ele pode:

- **Visualizar agendamentos** — consultar os atendimentos.
- **Atualizar status do atendimento** — por exemplo, marcar como confirmado, concluído ou cancelado.
- **Enviar notificações** — comunicar informações aos clientes.
- **Registrar observações** — adicionar informações relacionadas ao atendimento.

Isso resolve justamente a questão levantada pelo seu professor: **o sistema não fica limitado a apenas um administrador**.

No futuro, por exemplo, poderiam existir:

Administrador 1 → acesso completo
Administrador 2 → acesso completo
Operador 1 → funções operacionais
Operador 2 → funções operacionais
Operador 3 → funções operacionais

4. Autenticação

No centro do diagrama aparece **"Autenticar-se"**.

Essa função representa o processo de login do usuário.

Antes de acessar determinadas funcionalidades, o sistema verifica quem está tentando entrar e quais permissões essa pessoa possui.

Por exemplo:

Login → autenticação → identificação do tipo de usuário → acesso às funções permitidas.

Assim, um operador não precisa necessariamente ter as mesmas permissões de um administrador.

5. Validação dos dados

O caso de uso **"Validar dados"** representa as verificações realizadas pelo sistema antes de registrar informações.

Por exemplo:

- verificar se os campos obrigatórios foram preenchidos;
- verificar se o telefone está correto;
- verificar se o horário está disponível;
- verificar se as informações do agendamento são válidas.

6. Verificar disponibilidade

Quando o cliente tenta realizar um agendamento, o sistema precisa verificar se aquele horário realmente está disponível.

Por exemplo:

Cliente → escolhe serviço → escolhe data/horário → sistema verifica disponibilidade → agendamento.

Isso evita que duas pessoas sejam agendadas para o mesmo horário.

7. Registrar informações

Depois que os dados são preenchidos e validados, o sistema registra as informações necessárias para realizar o atendimento.

Esses dados posteriormente podem aparecer para o operador ou administrador na consulta dos agendamentos.

8. Gerar confirmação do agendamento

Depois que o agendamento é realizado, o sistema pode gerar uma confirmação.

O fluxo fica aproximadamente:

Cliente solicita agendamento → sistema valida informações → verifica disponibilidade → registra informações → gera confirmação.

9. Enviar notificações

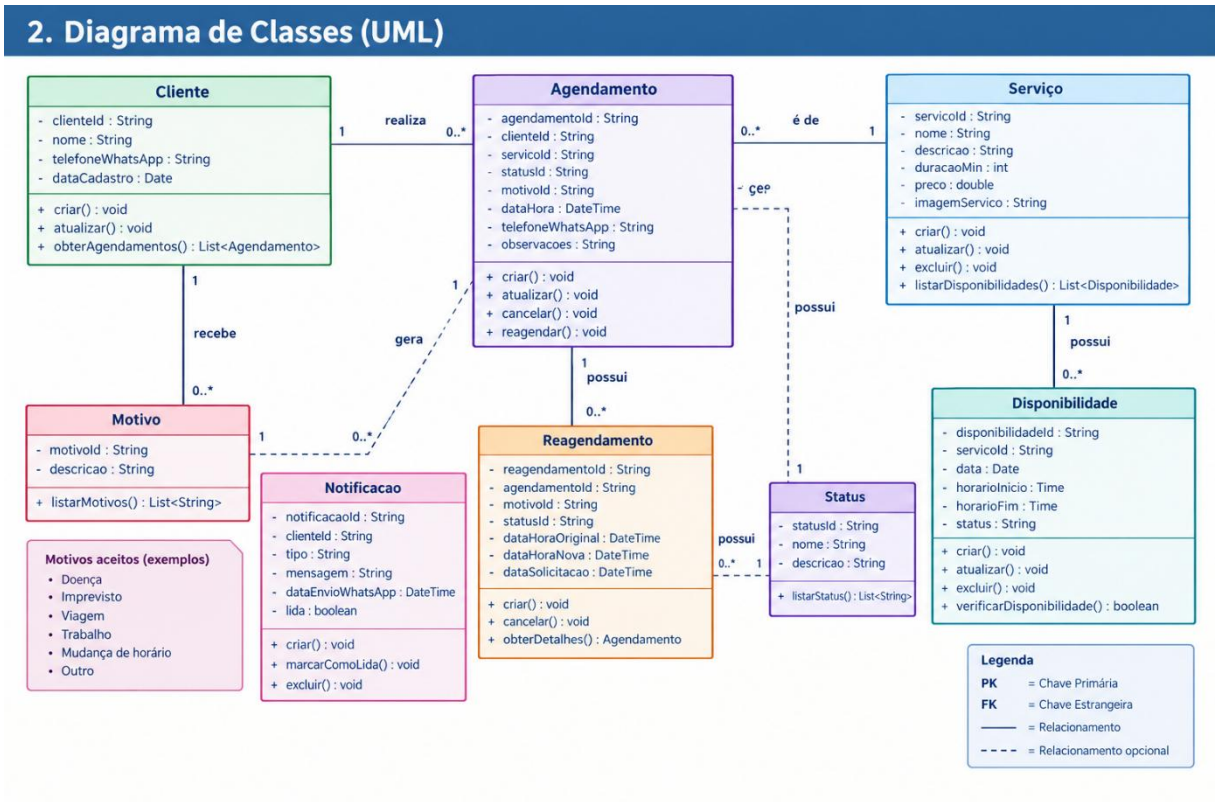
O sistema também possui a função de **enviar notificações**, que pode ser utilizada para comunicar o cliente sobre situações como:

- confirmação do agendamento;

- alteração de horário;
- reagendamento;
- cancelamento;
- lembrete do atendimento.

No diagrama, isso aparece relacionado às notificações por **WhatsApp/E-mail**

DIAGRAMA DE CLASSE



1. Cliente

A classe **Cliente** representa a pessoa que utiliza o sistema para realizar um agendamento.

Atributos:

- clienteId: identifica o cliente.
- nome: nome do cliente.
- telefoneWhatsApp: número utilizado para contato pelo WhatsApp.
- dataCadastro: data em que o cliente foi registrado.

Métodos:

- criar(): cadastra um cliente.
- atualizar(): altera os dados.
- obterAgendamentos(): consulta os agendamentos daquele cliente.

Relacionamento:

Um cliente pode realizar **zero ou vários agendamentos**, enquanto cada agendamento pertence a um cliente.

---

2. Agendamento

É a classe central do sistema. Representa o horário marcado pelo cliente para realizar determinado serviço.

Atributos:

- agendamentoId: identifica o agendamento.
- clienteId: identifica o cliente.
- servicoId: identifica o serviço escolhido.
- statusId: indica o status do agendamento.
- motivoId: identifica o motivo, quando necessário.
- dataHora: data e horário marcado.
- telefoneWhatsApp: contato utilizado pelo sistema.
- observacoes: informações adicionais.

Métodos:

- criar(): cria um agendamento.
- atualizar(): altera o agendamento.
- cancelar(): cancela.
- reagendar(): solicita ou realiza uma alteração de horário.

Relacionamentos:

- Cliente → Agendamento: **1 para 0..***
- Serviço → Agendamento: **1 para 0..***
- Agendamento → Status: utiliza um status.
- Agendamento → Motivo: pode utilizar um motivo.

---

3. Serviço

Representa os serviços oferecidos pelo estabelecimento.

Atributos:

- servicoId: identifica o serviço.
- nome: nome do serviço.
- descricao: descrição.
- duracaoMin: duração em minutos.

- preco: valor.
- imagemServico: imagem do serviço.

Métodos:

- criar()
- atualizar()
- excluir()
- listarDisponibilidades()

Um serviço pode possuir **vários horários de disponibilidade**.

---

4. Disponibilidade

Essa classe controla os horários em que determinado serviço pode ser agendado.

Atributos:

- disponibilidadeId: identifica a disponibilidade.
- servicoId: identifica o serviço.
- data: dia disponível.
- horarioInicio: horário inicial.
- horarioFim: horário final.
- status: informa se o horário está disponível ou não.

Métodos:

- criar()
- atualizar()
- excluir()
- verificarDisponibilidade()

Importante: aqui fica a informação de **data e horário de disponibilidade**, e não na tabela Serviço.

---

5. Reagendamento

Representa uma solicitação de alteração de um agendamento existente.

Atributos:

- reagendamentoId: identifica o reagendamento.
- agendamentoId: identifica o agendamento original.
- motivoId: identifica o motivo.
- statusId: indica o estado da solicitação.
- dataHoraOriginal: horário anteriormente marcado.
- dataHoraNova: novo horário solicitado.
- dataSolicitacao: quando o pedido foi realizado.

Métodos:

- criar(): registra o pedido.
- cancelar(): cancela o pedido.
- obterDetalhes(): consulta os detalhes.

Importante: retiramos o **telefone** daqui porque o contato já é registrado no cadastro/formulário do cliente.

---

6. Motivo

A classe **Motivo** serve para padronizar as justificativas de reagendamento.

Atributos:

- motivoId
- descricao

Os motivos aceitos podem ser:

- Doença
- Imprevisto
- Viagem
- Trabalho
- Mudança de horário
- Outro

Assim, em vez de cada pessoa escrever um motivo diferente, o sistema mantém opções padronizadas.

---

7. Status

A classe **Status** padroniza os estados utilizados pelo sistema.

Atributos:

- statusId
- nome
- descricao

Podem existir, por exemplo:

- Agendado
- Confirmado
- Concluído
- Cancelado
- Reagendado
- Pendente

Isso evita deixar status como um texto solto em várias partes do sistema.

---

8. Notificação

É responsável pelas mensagens enviadas ao cliente, principalmente pelo **WhatsApp**.

Atributos:

- notificacaoId
- clienteId
- tipo
- mensagem
- dataEnvioWhatsApp
- lida

Métodos:

- criar()
- marcarComoLida()
- excluir()

A dataEnvioWhatsApp registra **quando a mensagem foi enviada pelo WhatsApp**.